Assignment 5: Perceptron

MATH-CSCI 485

Piyush Jadhav

April 2026

Contents

Introduction

The **Perceptron** is one of the foundational models in machine learning, inspired by the biological neuron. Given an input vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, it computes a weighted linear combination followed by an activation function:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b \tag{1}$$

The output $\hat{y}$ depends on the choice of activation. In this assignment two variants are explored:

- **Part 1 — Heuristic:** continuous output via the sigmoid $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$, with weights updated on every point proportional to the prediction error.
- **Part 2 — Gradient Descent:** hard binary output $\hat{y} = \mathbf{1}[z \geq 0]$, with weights updated only on misclassified points by a fixed step equal to the learning rate.

Dataset

The dataset `data.csv` contains **100 samples**, each with two continuous features $(x_1, x_2) \in [0, 1]^2$ and a binary label $y \in \{0, 1\}$.

- **Class 0** (red dots): 50 samples clustered in the lower-left region.
- **Class 1** (blue dots): 50 samples clustered in the upper-right region.

The two classes are approximately linearly separable with a small number of borderline points, making this a suitable testbed for perceptron learning.

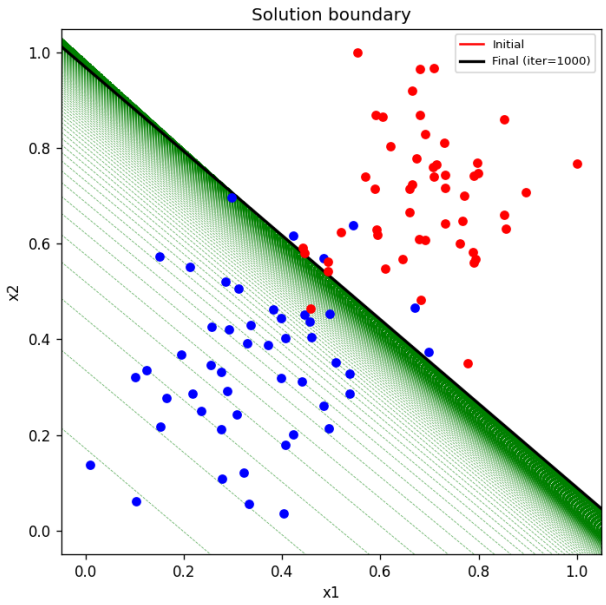


Figure 1: Raw data — blue dots are Class 1, red dots are Class 0. The classes are approximately linearly separable.

Part 1 — Heuristic Perceptron

Algorithm Description

The heuristic perceptron uses the **sigmoid activation** to produce a continuous prediction $\hat{y} \in (0, 1)$, which is then compared against the true binary label y to compute an error signal:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \text{error} = y - \hat{y} \tag{2}$$

The weights and bias are updated after every individual training point (online/stochastic learning):

$$b \leftarrow b + r\,(y - \hat{y}) \tag{3}$$

$$w_i \leftarrow w_i + r\,(y - \hat{y})\,x_i \quad \forall\, i \tag{4}$$

where r is the learning rate. Training repeats over all data points for up to a fixed maximum number of iterations, stopping early if the total absolute error falls below a tolerance $\epsilon = 10^{-4}$.

Decision Boundary

The decision boundary is the set of points satisfying $z = 0$, i.e. $w_1 x_1 + w_2 x_2 + b = 0$, which gives:

$$x_2 = -\frac{w_1 x_1 + b}{w_2} \tag{5}$$

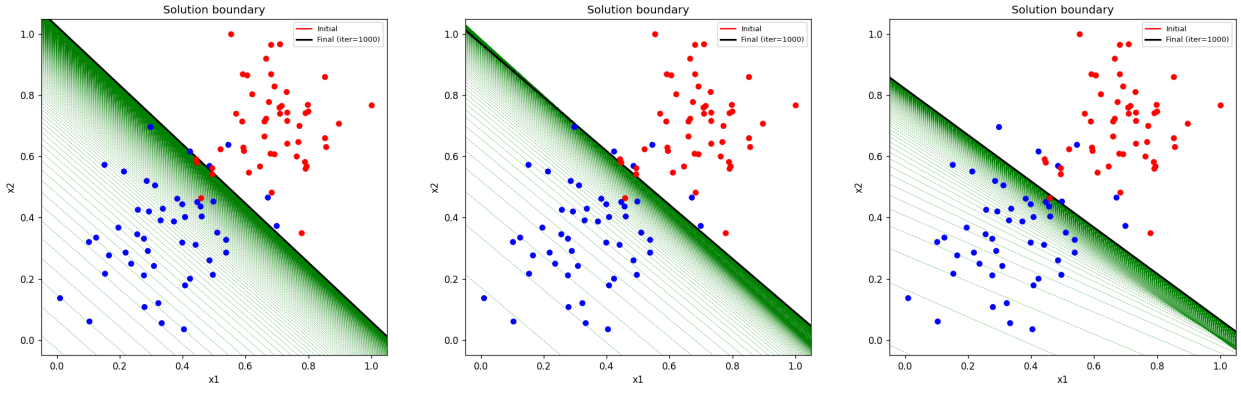
Implementation

Listing 1: Heuristic Perceptron Training Loop

```python
def perceptron_heuristic(lr, max_iter=1000, tol=1e-4, seed=42):
    np.random.seed(seed)
    w = np.random.randn(2) * 0.01
    b = np.random.randn()  * 0.01

    for it in range(max_iter):
        total_error = 0
        for xi, yi in zip(X, y):
            z    = np.dot(w, xi) + b
            yhat = sigmoid(z)        # continuous output
            err  = yi - yhat
            b   += lr * err          # bias update
            w   += lr * err * xi    # weight update
            total_error += abs(err)

        if total_error < tol:
            break  # early stopping
```

Results — Varying Learning Rate

Three learning rates were evaluated: $r \in \{0.01, 0.1, 1.0\}$ with a maximum of 1000 iterations. In each plot the **red** line shows the initial (random) decision boundary, **dashed green** lines show the boundary after each full pass through the data, and the **black** line is the final learned boundary.



(a) $r = 0.01$, 1000 iter (b) $r = 0.1$, 1000 iter (c) $r = 1.0$, 1000 iter

Figure 2: Part 1 — Heuristic Perceptron decision boundary evolution for three learning rates. Red = initial, dashed green = per-iteration updates, black = final boundary.

Analysis

Table 1: Summary of Part 1 results across learning rates.

Learning Rate	Iterations	Converged?	Observations
0.01	1000 (max)	No	Boundary barely moves; weight steps are too tiny
0.1	1000 (max)	No	Moderate sweep; boundary approaches the separating hy
1.0	1000 (max)	No	Wide oscillations early; sigmoid dampens instability

Key observations:

1. **Small learning rate ($r = 0.01$):** The green dashed lines are tightly clustered near the initial boundary. Weight updates are so small that the algorithm makes negligible progress within 1000 iterations. This demonstrates the risk of vanishing updates with very small learning rates.
2. **Moderate learning rate ($r = 0.1$):** The boundary sweeps across the data space in a controlled manner. The final black line achieves a visually reasonable separation, representing the best trade-off between convergence speed and stability.
3. **Large learning rate ($r = 1.0$):** The boundary oscillates widely in early iterations, visible as the broad spread of green dashes. However, because the update magnitude is proportional to the sigmoid error $|y - \hat{y}| \leq 1$, the updates remain bounded and the algorithm does not diverge. The final boundary is comparable to $r = 0.1$.
4. The dataset is **not perfectly linearly separable**, which explains why none of the three configurations converge within 1000 iterations. The sigmoid's continuous output means even well-classified points contribute a small nonzero gradient, preventing strict convergence.

Part 2 — Gradient Descent Perceptron

Algorithm Description

Part 2 uses a **hard threshold (step function)** for classification. A point is classified as Class 1 if $z \geq 0$, else Class 0. Weights are only updated when a point is *misclassified*:

$$\text{If predicted 0 but true 1}: \quad b \leftarrow b + r, \quad w_i \leftarrow w_i + r\,x_i \tag{6}$$

$$\text{If predicted 1 but true 0}: \quad b \leftarrow b - r, \quad w_i \leftarrow w_i - r\,x_i \tag{7}$$

Training runs for a fixed number of **epochs** (full passes over all data).

Log-Loss (Error Metric)

Although the update rule is binary, we track learning progress using **log-loss** (binary cross-entropy), computed with sigmoid probabilities:

$$\mathcal{L} = -\frac{1}{N}\sum_{j=1}^{N}\Big[y_j \log \sigma(z_j) + (1-y_j)\log(1-\sigma(z_j))\Big] \tag{8}$$

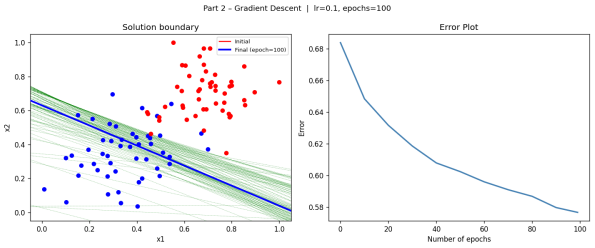
This gives a smooth, differentiable proxy for classification quality. Log-loss is recorded every 10 epochs and plotted as the error curve.

Implementation

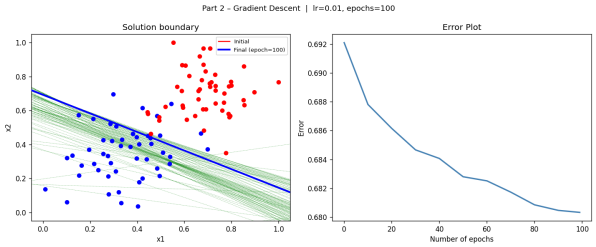
Listing 2: Gradient Descent Perceptron Training Loop

```
def perceptron_gradient_descent(lr, epochs=100, seed=42):
    np.random.seed(seed)
    w = np.random.randn(2) * 0.01
    b = np.random.randn()  * 0.01

    for epoch in range(epochs):
        for xi, yi in zip(X, y):
            z    = np.dot(w, xi) + b
            pred = 1 if z >= 0 else 0   # hard threshold
            if pred != int(yi):         # only update misclassified
                if pred == 0:
                    b += lr;  w += lr * xi
                else:
                    b -= lr;  w -= lr * xi

        # Record log-loss every 10 epochs
        if epoch % 10 == 0:
            errors.append(log_loss(X, y, w, b))
```

Results — Varying Learning Rate and Epochs

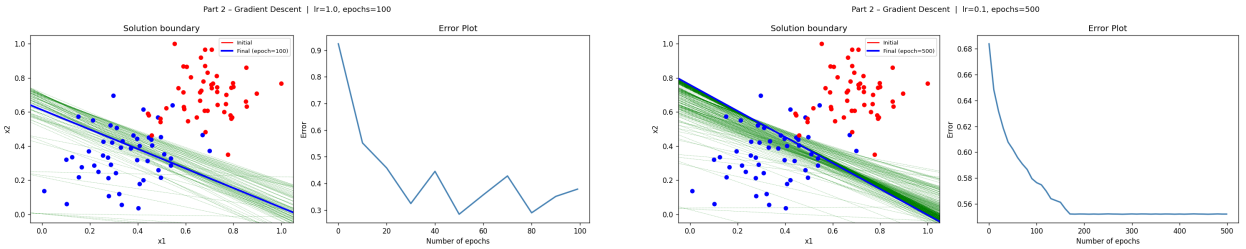


(a) $r = 0.1$, 100 epochs



(b) $r = 0.01$, 100 epochs

Figure 3: Part 2 results at moderate and small learning rates. Red = initial, dashed green = per-epoch boundaries, blue = final boundary. Right panel shows the error plot (log-loss vs. epochs).



(a) $r = 1.0$, 100 epochs (b) $r = 0.1$, 500 epochs

Figure 4: Part 2 results at large learning rate and extended training. The error plots show the characteristic steep-then-plateau shape.

Analysis

Table 2: Summary of Part 2 results across configurations.

r	**Epochs**	**Final Log-Loss**	**Notes**
0.01	100	≈ 0.680	Near-random; insufficient progress
0.1	100	≈ 0.577	Good convergence; stable error curve
1.0	100	≈ 0.379	Fastest improvement; aggressive updates
0.1	500	≈ 0.552	Marginal gain; plateau reached early

Key observations:

1. **Effect of learning rate:** A higher learning rate ($r = 1.0$) produces the steepest initial drop in log-loss and the lowest final error within 100 epochs. The boundary moves aggressively per epoch, correcting large numbers of misclassifications quickly.
2. **Small learning rate ($r = 0.01$):** With 100 epochs and tiny updates, the perceptron barely escapes the initial configuration. The final log-loss of ≈ 0.68 is close to the loss of a random classifier ($\ln 2 \approx 0.693$), confirming very little learning.
3. **Effect of epoch count:** Extending from 100 to 500 epochs at $r = 0.1$ yields only marginal improvement ($0.577 \to 0.552$). The error curve plateaus because the data is not perfectly linearly separable; the boundary cannot converge to a perfect solution, so further iterations cycle near the same local minimum.
4. **Error curve shape:** In all cases the log-loss curve exhibits a characteristic **exponential decay followed by a plateau** — rapid initial learning as obvious misclassifications are corrected, then diminishing returns as the remaining borderline points are structurally difficult to separate with a linear boundary.
5. **Boundary evolution:** With $r = 1.0$ the green dashes span a large region, reflecting the large steps. With $r = 0.01$ the dashes are nearly identical to one another, reflecting almost no movement.

Comparison: Part 1 vs Part 2

Table 3: Conceptual comparison between the two perceptron approaches.

Property	**Part 1 (Heuristic)**	**Part 2 (GD)**
Activation	Sigmoid (continuous)	Step function (binary)
Update trigger	Every data point	Misclassified points only
Update magnitude	$r \cdot \lvert y - \hat{y} \rvert$	Fixed: r
Error signal	$y - \sigma(z) \in (-1, 1)$	± 1 (sign only)
Stability	Smoother (bounded error)	Can oscillate at high r
Convergence	Soft; depends on tolerance	Hard; stops when 0 errors
Error metric	Total absolute sigmoid error	Log-loss (cross-entropy)

The heuristic approach is gentler: because the update magnitude is bounded by $|y - \hat{y}| < 1$, even a large learning rate cannot produce arbitrarily large weight changes. The GD approach applies a fixed-magnitude correction, which can cause the boundary to oscillate substantially with high learning rates, but also means it corrects confidently-wrong predictions as decisively as confidently-right ones.

Conclusion

Both perceptron variants successfully learned a decision boundary that approximately separates the two classes in the 2D dataset. The main findings are:

- **Learning rate** is the most critical hyperparameter: too small ($r = 0.01$) means negligible progress within practical iteration limits; too large can cause oscillation, though the sigmoid in Part 1 provides a self-limiting mechanism.
- A **moderate learning rate** ($r \approx 0.1$) consistently provides the best balance of convergence speed and stability across both methods.
- **Increasing epochs** offers diminishing returns once the error curve plateaus, which occurs because the data is not perfectly linearly separable. In such cases, more powerful models (e.g., multi-layer perceptrons with nonlinear activations) would be needed.
- The **log-loss curve** in Part 2 is a useful diagnostic: a steep initial drop followed by a plateau indicates the perceptron has exhausted the easily-correctable misclassifications and has reached the structural limit of a linear classifier on this dataset.